

# **Smart contract audit**

## **rain.datacontract**

# Content

|                                                          |          |
|----------------------------------------------------------|----------|
| <b>Project description</b>                               | <b>3</b> |
| <b>Executive summary</b>                                 | <b>3</b> |
| <b>Reviews</b>                                           | <b>3</b> |
| <b>Technical analysis and findings</b>                   | <b>4</b> |
| <b>Security findings</b>                                 | <b>5</b> |
| **** Critical                                            | 5        |
| *** High                                                 | 5        |
| ** Medium                                                | 5        |
| M01 - Silent Failure in writeZoltu When Proxy Is Missing | 5        |
| * Low                                                    | 5        |
| L01 - Off-by-One Memory Read in write Function           | 5        |
| Informational                                            | 6        |
| I01 - Stale or Misleading Assembly Comments              | 6        |
| I02 - Hardcoded Deterministic Deployment Proxy Address   | 7        |
| <b>General Risks</b>                                     | <b>8</b> |
| <b>Approach and methodology</b>                          | <b>9</b> |



## Project description

This repository contains a Solidity library titled LibDataContract, which is a simplified, gas-optimized reimplementation of SSTORE2. It provides functionality for writing arbitrary data to contract bytecode and reading it back, enabling efficient on-chain data storage without the overhead of standard storage slots. The library features assembly optimizations for minimal gas usage, implements strict safety checks for out-of-bounds slicing, and includes support for deterministic deployment across networks. It is designed to be a minimal, zero-dependency foundation for handling bytecode-stored data.

## Executive summary

|                      |                                                                                                                   |
|----------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Type</b>          | Library                                                                                                           |
| <b>Languages</b>     | Solidity                                                                                                          |
| <b>Methods</b>       | Architecture Review, Manual Review, Unit Testing, Functional Testing, Automated Review                            |
| <b>Documentation</b> | README.md                                                                                                         |
| <b>Repositories</b>  | <a href="https://github.com/rainlanguage/rain.datacontract">https://github.com/rainlanguage/rain.datacontract</a> |

## Reviews

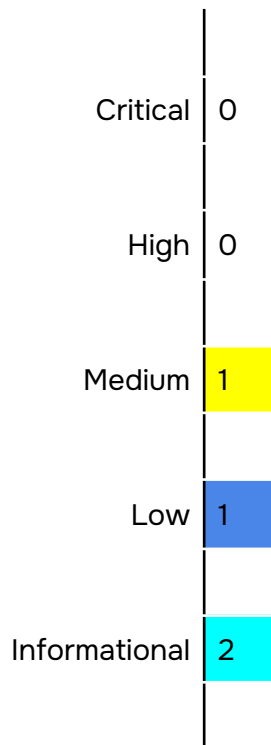
| Date     | Repository        | Commit                                   |
|----------|-------------------|------------------------------------------|
| 26/01/26 | rain.datacontract | 061bf7cd63496ccbe4f74bcd0c48715883c1c3cb |
| 28/01/26 | rain.datacontract | 6217921940ff7b2f0bd260d79573b109a2f8bf66 |

## Scope

|                               |
|-------------------------------|
| <b>Contracts</b>              |
| src/lib/LibDataContract.sol   |
| src/error/ErrDataContract.sol |



## Technical analysis and findings



# Security findings

## \*\*\*\* Critical

No critical severity issue found.

## \*\*\* High

No high severity issue found.

## \*\* Medium

### M01 - Silent Failure in writeZoltu When Proxy Is Missing

The `writeZoltu()` function performs a low-level call to a hardcoded deterministic deployment proxy address (0x7A0D...). It relies on the success boolean returned by the call instruction to detect failures.

However, in the EVM, a low-level call to an address that contains no code (i.e., if the proxy is not deployed on the current chain) returns true (success) but executes nothing and returns no data. In this scenario:

1. The call returns success = true.
2. The `mload(0)` reads 0 (since no data was returned).
3. The if (!success) check passes.
4. The function returns `address(0)` effectively silently failing.

This differs from the standard write function, which explicitly checks if `(pointer == address(0))` to detect deployment failures.

**Path:** `src/lib/LibDataContract.sol`

**Recommendation:** Update the validation logic in `writeZoltu` to require that the returned address is non-zero. This ensures the deployment actually occurred.

**Found in:** 061bf7cd

**Status:** Fixed at 94159c97

## \* Low

### L01 - Off-by-One Memory Read in write Function

The `write()` and `writeZoltu()` functions calculate the payload size for the create/call opcodes by adding `prefixLength` (13) to the data length stored in the prefix.

The issue arises from "double counting" the 0x00 separator byte:



1. Physically: The 0x00 byte is already included within the 13-byte prefixLength.
2. Logically: The length stored in the prefix (calculated as `user_data_length + 1` in `newContainer`) also accounts for this 0x00 byte to satisfy the contract deployment logic.

When these are added together (`prefixLength + storedLength`), the 0x00 byte is effectively counted twice, causing the calculated size to be 1 byte larger than the actual populated memory.

As a result, the create opcode reads one byte past the end of the valid data. While Solidity's 32-byte memory alignment ensures this extra byte is read from zero-padded reserved memory (preventing dirty reads), the logic is technically incorrect and consumes a negligible amount of extra gas.

**Path:** `src/lib/LibDataContract.sol`

**Recommendation:** Subtract 1 from the size calculation in `write` and `writeZoltu` to correct the double counting.

**Found in:** 061bf7cd

**Status:** Fixed at d8b8fa3b

## Informational

### I01 - Stale or Misleading Assembly Comments

The documentation block above the `BASE_PREFIX` constant details an assembly sequence based on the original `SSTORE2` implementation. It lists opcodes, such as 0x63 (`PUSH4`), and offsets, like 0x0e. However, these do not match the actual optimized bytecode implemented in `BASE_PREFIX`, which uses 0x61 (`PUSH2`) and an offset of 0x0c.

While the comments include a note explaining the optimization ("Note also that we don't need 4 bytes..."), the primary visual opcode map contradicts the code immediately following it. This can confuse developers or auditors trying to verify the bytecode construction manually.

**Path:** `src/lib/LibDataContract.sol`

**Recommendation:** Update the opcode breakdown in the comments to exactly match the values in `BASE_PREFIX` (showing 0x61 and 0x600c). If the comparison to the original `SSTORE2` implementation is important for context, clearly label the existing



chart as "Reference Implementation" and add a corrected chart labeled "Current Implementation."

**Found in:** 061bf7cd

**Status:** Fixed at 00b0fa01

## I02 - Hardcoded Deterministic Deployment Proxy Address

The *writeZoltu()* function performs a low-level call to a hardcoded deterministic deployment proxy address: 0x7A0D94F55792C434d74a40883C6ed8545E406D12.

This address is embedded directly in the function body and cannot be modified or configured.

Hardcoding an external dependency inline inside function logic introduces some maintainability and integration risks:

- Critical configuration is hidden inside implementation details,
- The dependency may be easy to overlook during reviews and integrations.

**Path:** src/lib/LibDataContract.sol

**Recommendation:** Move the proxy address into a named constant variable with clear documentation.

**Found in:** 061bf7cd

**Status:** Fixed at 18eb6419



## General Risks

- **Contract Size Limit (EIP-170):** The Ethereum Virtual Machine (EVM) imposes a limit on contract size (typically 24KB). Attempting to write a data blob that, combined with the deployment prefix, exceeds this limit will cause the write transaction to fail. Users must ensure their data fits within these constraints.
- **Immutability:** Data stored via this method is immutable. Unlike standard storage slots which can be updated, updating data stored in bytecode requires deploying a completely new contract and updating any references to it in the consuming system.
- **Deployment Cost:** While reading this data is gas-efficient, the initial cost to deploy the data contract scales linearly with the size of the data. Large datasets may incur significant gas costs during the write operation.





## Approach and methodology

To establish a uniform evaluation, we define the following terminology in accordance with the OWASP Risk Rating

Methodology:

|                                                                                   |                                                                                                                                                                                  |
|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <b>Likelihood</b><br>Indicates the probability of a specific vulnerability being discovered and exploited in real-world scenarios                                                |
|  | <b>Impact</b><br>Measures the technical loss and business repercussions resulting from a successful attack                                                                       |
|  | <b>Severity</b><br>Reflects the comprehensive magnitude of the risk, combining both the probability of occurrence (likelihood) and the extent of potential consequences (impact) |

Likelihood and impact are divided into three levels: High H, Medium M, and Low L. The severity of a risk is a blend of these two factors, leading to its classification into one of four tiers: Critical, High, Medium, or Low.

When we identify an issue, our approach may include deploying contracts on our private testnet for validation through testing. Where necessary, we might also create a Proof of Concept PoC to demonstrate potential exploitability. In particular, we perform the audit according to the following procedure:

|                                                                                     |                                                                                                                                                                                                                 |
|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <b>Advanced DeFi Scrutiny</b><br>We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs                           |
|  | <b>Semantic Consistency Checks</b><br>We then manually check the logic of implemented smart contracts and compare with the description in the white paper.                                                      |
|  | <b>Security Analysis</b><br>The process begins with a comprehensive examination of the system to gain a deep understanding of its internal mechanisms, identifying any irregularities and potential weak spots. |